

Deep learning / apprentissage profond

Cours 3 : réseaux convolutifs

Hervé Le Borgne
herve.le-borgne@cea.fr

2019-2020

Rappels ($\rightarrow$ pré-requis)

- apprentissage supervisé : principes et détails
- perceptron multi couche
- principes et méthodes d'apprentissage des réseaux profonds
 - ▶ maximum de vraisemblance
 - ▶ définition fonction de coût
 - ▶ descente de gradient (stochastique)
 - ▶ rétro-propagation du gradient
- couches cachées et non linéarités
- intérêt de l'apprentissage profond

Plan

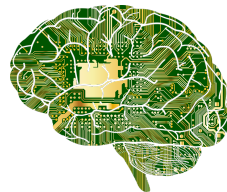
- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Remarques préliminaires

- Les réseaux convolutifs traitent des données intrinsèquement organisées selon une grille :
 - ▶ signaux échantillonnés (séries temporelles...)
 - ▶ images
- Ils sont actuellement les « réseaux profonds » les plus employés et ceux ayant conduit à certaines des applications les plus spectaculaires
- En particulier, ils ont été à l'origine de l'engouement de la « troisième vague connexionniste » (*deep learning*) faisant suite à :
 - ▶ des avancées théoriques pour faire converger de grandes architectures [Hinton *et al.*, 2006]
 - ▶ une croissance exponentielle des données disponibles (*smart phone*, web 2.0...)
 - ▶ une puissance de calcul (GPGPU) adaptée à leur architecture



Remarques préliminaires

- État de l'art significativement amélioré dans des domaines dont le matériau de base est proche du signal :
 - ▶ en classification supervisée
 - ▶ pour « domaines proche signal »
 - ★ reconnaissance de parole (représentation temps-fréquence)
 - ★ vision par ordinateur (pixels)
- Moins spectaculaires pour le « traitement sémantique » (TAL)
 - ▶ Sauf pour la traduction (mais traitement statistique → quasi signal?)
- Ils ont une « inspiration biologique » et nourrissent l'analogie avec le cerveau des vertébrés (pas seulement humains...)



What color are her eyes?
What is the mustache made of?



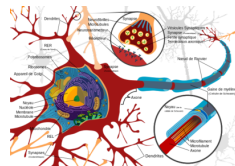
How many slices of pizza are there?
Is this a vegetarian pizza?



Is this person expecting company?
What is just under the tree?



Does it appear to be tasty?
Does the person have 20/20 vision?



Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

On les appelle :

- *Convolutional (neural) networks* → **CNN**
- Réseaux (de neurones) convolutifs
 - ▶ et à tort réseaux « convolutionels »
- **ConvNets!**

Définition

Un CNN est un réseau de neurones dont au moins une couche utilise la convolution en place du produit matriciel usuel (MLP « complètement connectés »)

- il existe des réseaux « purement » convolutifs
- ils sont couramment suivis de couches MLP « complètement connectées » (*fully connected*)
 - ▶ Mais ces couches peuvent être vues comme des convolutions aussi!

Produit de convolution

Si les fonctions f et ω sont intégrables (au sens de Lebesgues) sur $\mathbb{R}$, leur produit de convolution est :

$$(f * \omega)(x) = \int_{-\infty}^{+\infty} f(t)\omega(x - t) dt \quad (1)$$

Pour des fonctions discrètes (suites) :

$$(f * \omega)(n) = \sum_{m=-\infty}^{m=+\infty} f(m)\omega(n - m) \quad (2)$$

Version 2D :

$$(f * \omega)(x, y) = \int_{-\infty}^{+\infty} f(t, u)\omega(x - t, y - u) dt du \quad (3)$$

Vocabulaire

$$(f * \omega)(x) = \int f(t)\omega(x - t) dt \quad (4)$$

- f est le signal (image) d'entrée (*input*)
- ω est un noyau (*kernel*)
 - ▶ « filtre » « masque de convolution »
- $f * \omega$ est une carte d'activation (*feature map*)
- À noter :
 - ▶ les entrées sont généralement multidimensionnelles (e.g. une image a trois couches R,G,B). On considère alors des tenseurs.
 - ▶ les entrées et noyaux sont considérés comme nuls en dehors du support stocké effectivement ($\rightarrow \int_{-\infty}^{+\infty}$)

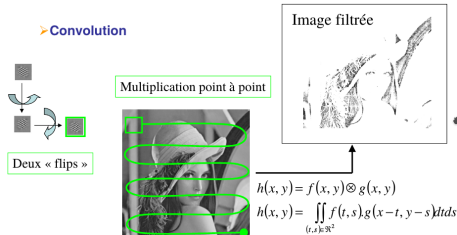
Interprétation

$$(f * \omega)(x) = \int f(t)\omega(x - t) dt$$

En 1D, si ω est constante sur un intervalle $[a, b]$, nous obtenons la **moyenne glissante** de f .



En 2D, si ω est un noyau (*kernel*) on obtient le **filtrage** de l'image f .
Le noyau ω est renversé (*flip*) droite-gauche et haut-bas



Propriétés

Commutativité :

$$(f * g)(x) = (g * f)(x) \quad (5)$$

Distributivité

$$(f * (g + h))(x) = (f * g)(x) + (f * h)(x) \quad (6)$$

Associativité

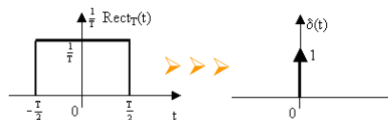
$$((f * g) * h)(x) = (f * (g * h))(x) \quad (7)$$

→ L'ensemble des fonctions intégrables munies de $(+, *)$ est un anneau non unitaire (sans élément neutre pour $*$) ou pseudo-anneau

Propriétés (2)

- Une impulsion de Dirac est une distribution ($\int_{-\infty}^{+\infty} \delta(t) = 1$) qui peut être vue comme la limite d'une fenêtre rectangulaire :

$$\delta(t) = \lim_{T \rightarrow 0} \frac{1}{T} \text{Rect}_T(t)$$



- Le Dirac est l'« élément neutre » de la convolution : $(f * \delta)(x) = f(x)$
- Dualité entre convolution temporelle/spatiale et multiplication fréquentielle
 - ▶ $\mathcal{F}$ étant la transformée de Fourier $\mathcal{F}(f) = \int_{-\infty}^{+\infty} f(t)e^{-2i\pi\omega t} dt$
 - ▶ f et g étant de carré intégrable (pour la seconde formule seulement)

$$\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g) \quad \text{et} \quad f * g = \mathcal{F}^{-1}(\mathcal{F}(f) \cdot \mathcal{F}(g))$$

Convolution en pratique dans les CNN

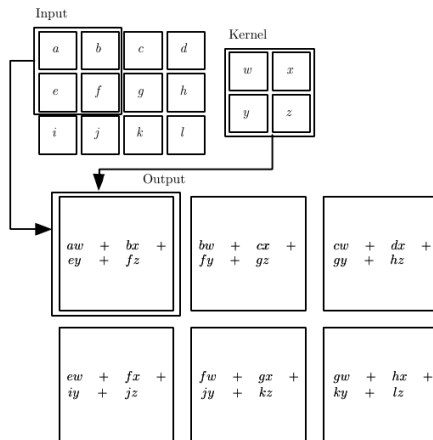
Spoiler : les réseaux convolutifs ne comportent généralement pas de convolution !

En pratique, les noyaux ne sont pas inversés (*flips*). On a alors la **cross-corrélation** :

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) \quad (8)$$

- L'inversion ne sert qu'à obtenir la propriété de commutativité, qui n'est pas utile en pratique.
 - ▶ dans un CNN, les convolutions sont généralement enchaînées avec des opérations non commutatives
- Par convention, on utilise quand même le terme « convolution »...

Calcul pratique de la convolution

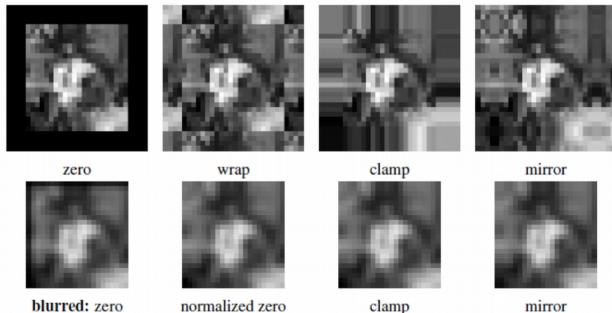


Taille $(3 \times 4) * (2 \times 2)$ donne $(2 \times 3) \rightarrow$ Gestion des bords ?

Calcul pratique de la convolution

Problème aux bords

- Mettre à 0 ou à une valeur constante (*zero padding*).
- Convolution partielle sur une portion du masque.
- Miroir de l'image : $f(-x, y) = f(x, y)$.
- Pas de solution miracle.

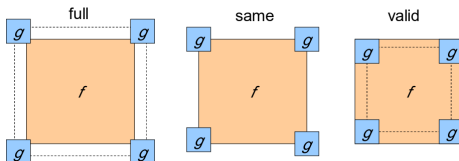


Source : Szeliski

Zero padding

- Trois types notoires :

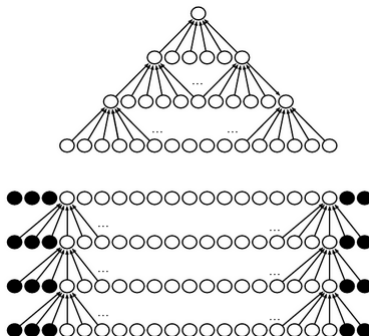
- ▶ *valid* : pas de padding ; noyau entier dans l'image
 - ★ tous les pixels de sortie ont la même validité statistique (car calculés à partir du même nombre d'entrées) → la taille de la sortie diminue
 - ★ limite donc la profondeur du réseau ! (feature map tendent vers 1 !)
- ▶ *same* : padding balancé de la taille du noyau
 - ★ même taille en entrée et sortie → profondeur non limitée
 - ★ pixels de bords sont moins significatifs
- ▶ *full* : tous les pixels ont même poids lors de la convolution ;
 - ★ chaque pixel d'entrée est « visité » le même nombre de fois par le noyau → la sortie « grossit »
 - ★ pixels de bords moins significatifs → nuisible à invariance en translation



source : S. Lazebnik

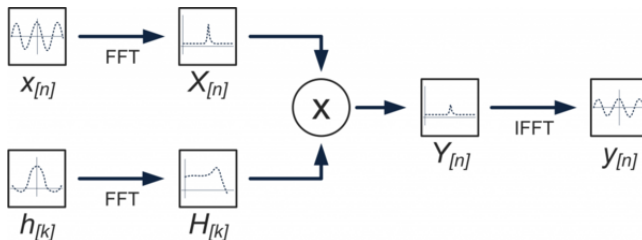
Zero padding

- en général, pour les CNN, il faut choisir entre *valid* et *same*
- exemple noyau de taille 6 à chaque couche
 - ▶ pas de padding (*valid*) : la taille diminue
 - ★ - 5 pix à chaque couche → trois couches max (pour 16 pixels en entrée)
 - ▶ en ajoutant 5 zéros (*same*) : largeur constante



Implémentation pratique

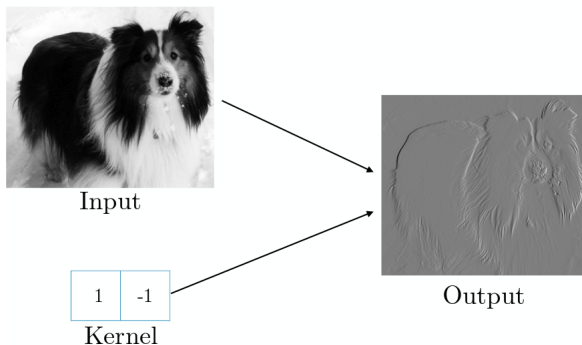
- CNN particulièrement adaptés au calcul massivement parallèle
 - ▶ GPU : *Graphics Process Units*
 - ▶ GPGPU : *general purpose GPU* [Simard et al., 2005]
- selon la taille image/noyau, calculer dans l'espace de Fourier peut être plus efficace



source: hamiltonkibbe.com

Exemple : filtrage

La convolution est une opération de *filtrage*. Exemple de détection de bord (opérateur différentiel) :



→ détection des bords (verticaux)

Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - **Propriétés remarquables**
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Propriétés remarquables

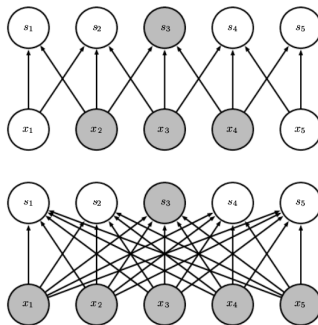
Le succès des CNN est partiellement dû :

- à leurs interactions parcimonieuses
→ efficacité calculatoire
- au partage des paramètres
→ gain mémoire
→ robustesse statistique
- à la propriété d'équivariance de représentation
→ invariance en translation

Interactions parcimonieuses

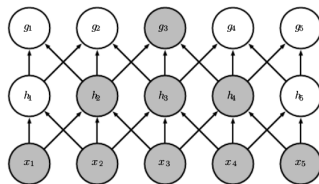
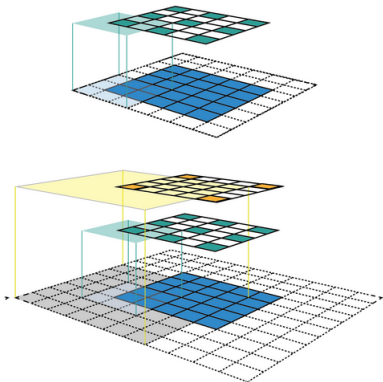
Vocabulaire : *sparse interactions*, *sparse connectivity* ou *sparse weights*

- Les couches MLP sont totalement connectées (*fully connected*)
 - ▶ Chacune des m entrées interagit avec chacune des n sorties
 - ▶ $m \times n$ paramètres à estimer (et complexité $O(m \times n)$)
- Les couches *conv* limitent à k connexions avec $k \ll m, n$
 - ▶ moins de paramètre à estimer – gain d'espace mémoire
 - ▶ estimation statistique plus robuste (à nombre de données équiv.)



Interactions parcimonieuses

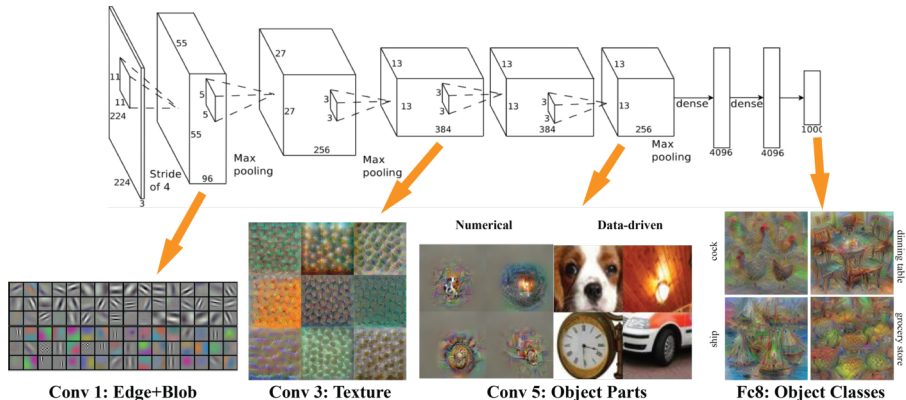
- La profondeur (*depth*) d'un CNN permet une **interaction indirecte** de filtres convolutifs avec de plus larges portions de l'entrée
 - ▶ Les premiers blocs décrivent des interactions parcimonieuses locales
 - ▶ Les blocs suivants décrivent des interactions de complexité croissante



NB : cela résulte aussi du *pooling* et des *strides* (cf. plus loin)

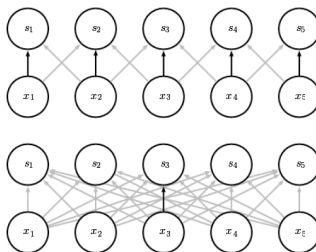
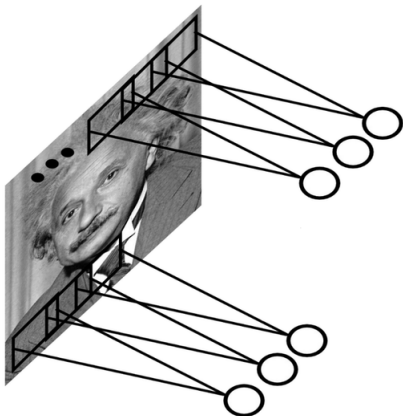
Interactions parcimonieuses

- sur les images, les interactions de complexité croissante portent une sémantique



Partage des paramètres

- MLP : chaque paramètre évalué indépendamment
- ConvNets : un paramètre de noyau est partagé à de multiples localisations de l'image
 - ▶ les propriétés statistiques *locales* sont réutilisables ailleurs
- → accroît la signifiante statistique de l'estimation



source [Ranzato, CVPR 2012] [Goodfellow et al, 2016]

Partage des paramètres et équivariance à la translation

Équivariance

La fonction f est équivariante à g si :

$$f(g(x)) = g(f(x))$$

Propriétés

La convolution est équivariante à la translation.

La convolution (en fait : cross corrélation) de I par K est (en 1D pour simplifier notations) :

$$(I * K)(x) = \sum_u I(u)K(u - x) \quad (9)$$

Si on translate le résultat de t (pixels) :

$$(I * K)(x - t) = \sum_u I(u)K(u - x + t) \quad (10)$$

Partage des paramètres et équivariance à la translation

Si on translate le résultat de t (pixels) :

$$(I * K)(x - t) = \sum_u I(u)K(u - x + t) \quad (11)$$

Soit L_t la translation de t (pixels). $[L_t I](x) = I(x - t)$

$$([L_t I] * K)(x) = \sum_u I(u - t)K(u - x) \quad (12)$$

$$= \sum_{u'} I(u')K(u' + t - x) \quad (13)$$

$$= (I * K)(x - t) \quad (14)$$

$$= [L_t [I * K]](x) \quad (15)$$

→ Translater puis convoluer est équivalent à convoluer puis translater

Propriété

La convolution est équivariante à la translation

Translater puis convoluer est équivalent à convoluer puis tradater

→ détection d'un motif localisé indépendamment du temps (séries temporelles) ou de la position (images)



NON Propriété

La convolution N'est PAS équivariante à la rotation, au zoom, etc.

→ nécessité de traitements spécifiques pour gérer ces transformations

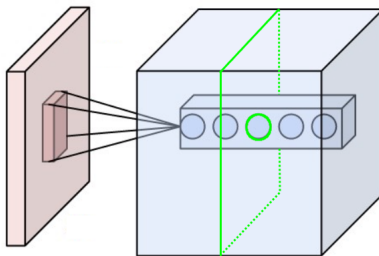
- traitement des données en entrée pour les rendre invariantes
 - ▶ ex : chercher l'axe principal de l'image puis la redresser pour le mettre vertical (→ invariance à la rotation)
- alimenter le CNN avec des données transformées
 - ▶ Cf. plus loin : augmentation de données + *pooling*



Plan

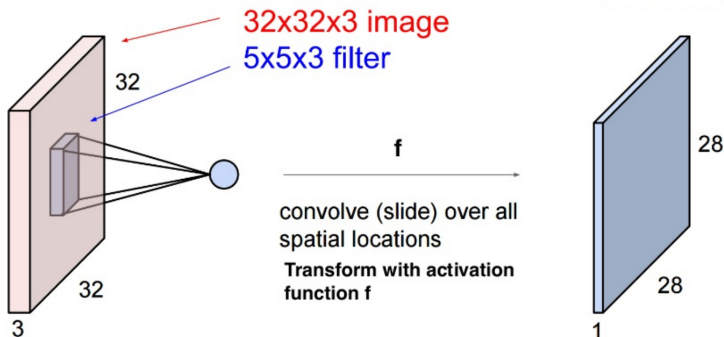
- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

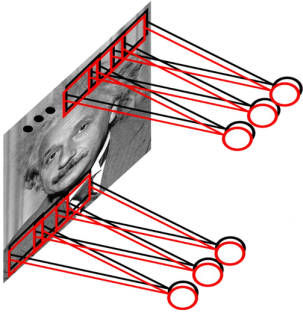
- la convolution avec *un* noyau extrait *un* type de caractéristique
- pour des *réseaux* de neurones, on cherche *plusieurs* caractéristiques
→ plusieurs noyaux en parallèle
- une couche (*layer*) comporte des neurones organisés en 3D :
 - ▶ arrangement spatial (hauteur×largeur) partageant les paramètres
 - ▶ empilement de cartes d'activations en profondeur



On détaille... [source](#) : Karpathy

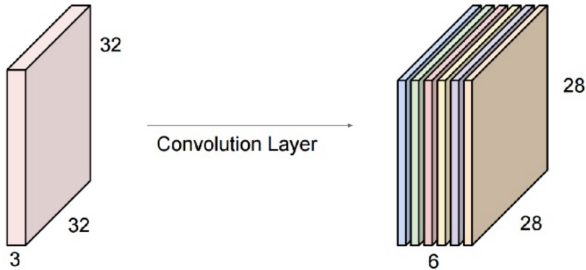
- image convoluée par noyau → carte d'activation (*feature map*)
- L'étendue spatiale du noyau est le champ récepteur (*receptive field*)
 - ▶ il est limité en hauteur/largeur (5)
 - ▶ il occupe toute la profondeur (e.g 3) du volume d'entrée





- une couche de convolution apprend plusieurs noyaux en parallèle
 - ▶ initialisés différemment, ils apprennent des choses différentes
(pour faire mieux décroître la *loss* lors de l'apprentissage)

- aboutit à un « empilement » de cartes d'activations



Calcul convolution en détail

- La sortie d'une couche de convolution est un volume 3D de valeurs
- Elle sert d'entrée à la couche suivante
- En pratique, la première couche de données est souvent un volume
 - ▶ une image est un volume $H \times W \times 3 = H \times W \times (R, G, B)$
 - ▶ la profondeur d'un tel volume est appelé canal (*channel*)
- Soit
 - ▶ une entrée (3D) $V_{i,j,k} = V_{canal_{in}, lig, col}$
 - ▶ des noyaux (4D) $K_{a,b,c,d} = K_{canal_out, canal_in, offset_lig, offset_col}$
(on considère $a = canal_out$ noyaux en parallèle)
 - ▶ une sortie (3D) $Z_{i,j,k} = Z_{canal_{out}, lig, col}$

Alors la sortie s'exprime par :

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n} \quad (16)$$

On détaille...

Calcul convolution en détail

- Soit

- ▶ une entrée $V_{canal_{in},lig,col} = V_{i,j,k}$
- ▶ une sortie $Z_{canal_{out},lig,col} = Z_{i,j,k}$
- ▶ des noyaux (4D) $K_{a,b,c,d} = K_{canal_out,canal_in,offset_lig,offset_col}$

Alors la sortie s'exprime par :

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n} \quad (17)$$

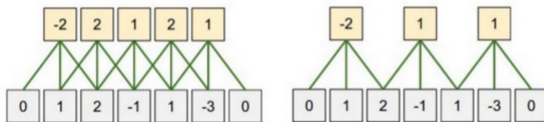
- nombre sorties ($Z_{i,j,k}$) = nombre canaux sortie ($K_{i,l,m,n}$)
- on moyenne sur tous les canaux d'entrée $\sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$
- multiplication (terme à terme) et accumulation (somme) (MAC : *multiply-accumulate*) sur tout le champs récepteur :

$$\sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$

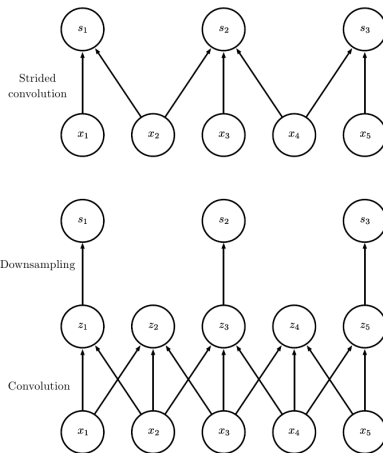
Convolution incomplète (*stride*)

- On peut « éviter » certaines positions spatiales
 - diminue le coût calculatoire
 - estimation moins précise de la caractéristique
- « équivalent » à un sous-échantillonnage de la sortie complète
- on définit un pas (*stride*) s

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,(j-1) \times s + m, (k-1) \times s + n} K_{i,l,m,n} \quad (18)$$

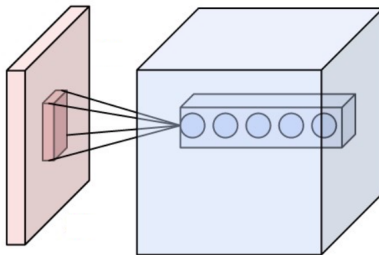


- Le pas s peut être différent selon la direction : s_{lig} , s_{col}



- la convolution avec un pas de $s = 2$ donne la même sortie que la convolution complète ($s = 1$) suivi d'un sous-échantillonnage
- mais les calculs sont moindres...

Exercice



Reconnaître/savoir situer

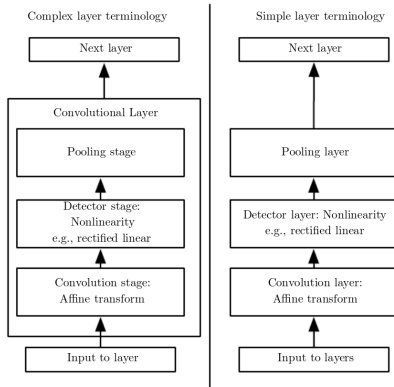
- Les paramètres/poids (partagés) appris
- Les cartes d'activation
- Le champs récepteur

Une couche CNN typique :

- convolution : transformation linéaire aboutissant à des cartes d'activation
- non linéarité : étape de détection (*detector stage*)
- agrégation (*pooling*) dans un voisinage (spatial)

Il existe **deux terminologies** de
« couches »

- complexes : couche à multiples étapes (*stage*). Elles correspondent à un « noyau complet »
- simples : chaque traitement. Certaines couches n'ont alors pas de paramètres !



Implémentation

- produits scalaires entre filtres et régions locales de l'entrée
 - ▶ régions de même taille D que les filtres
 - ▶ régions échantillonnées selon les *strides*
- régions et filtres transformés en vecteurs (i.e si ce st des matrices/tenseurs)
- on multiplie les deux matrices résultantes :
 - ▶ entrées : $D \times nb_regions$
 - ▶ filtres : $nb_filtres \times D$
- sortie = réponse de chaque filtre en chaque localisation
 - ▶ peut être « retaillée » en tenseur

Ex : image $227 \times 227 \times 3$ et 96 filtres $11 \times 11 \times 3$; stride 4

- on a $(227 - 11)/4 + 1 = 55$ localisations par lig/col $\rightarrow$ 3025 régions
- $D = 11 * 11 * 3 = 363$ donc entrée 363×3025 , filtres 96×363
- sortie 96×3025 retaillée en $55 \times 55 \times 96$ ([source](#) : Stanford CS231)

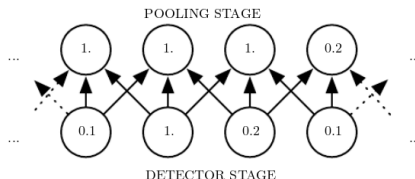
Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

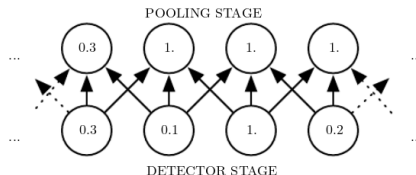
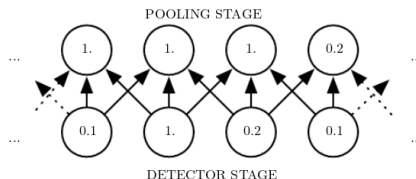
Agrégation (pooling)

L'agrégation « résume statistiquement » un voisinage

- En sortie d'une couche chaque valeur est remplacée par un « résumé » d'un voisinage spatial
 - ▶ Ex. de voisinage : fenêtre rectangulaire de petite taille (2×2 ; 3×3 ...)
- plusieurs types de *pooling* :
 - ▶ *max pooling* : valeur maximale du voisinage
 - ▶ *average pooling* : valeur moyenne du voisinage
 - ▶ *L_2 norm* : $(\sum_{neigh.} x_i^2)^{1/2}$
 - ▶ *weighed avg pooling* : moyenne pondérée selon distance au centre



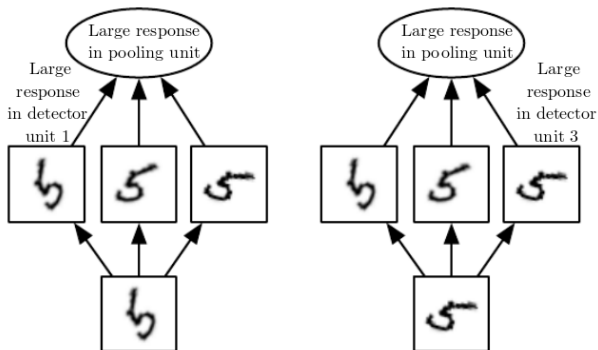
- permet une relative invariance aux petites translations



- L'importance de la localisation spatiale précise dépend de l'application.

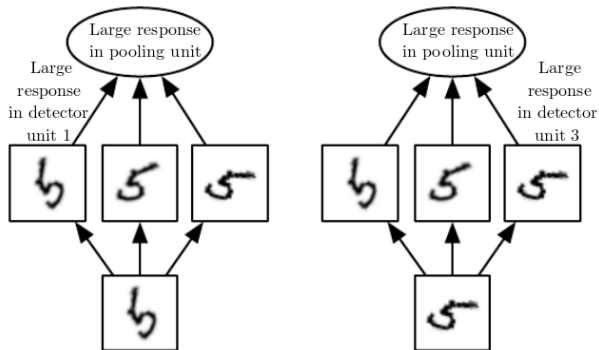
L'agrégation des sorties de plusieurs couches de convolution permet d'apprendre des invariants complexes.

Exemple de la rotation [Goodfellow et al., 2013]



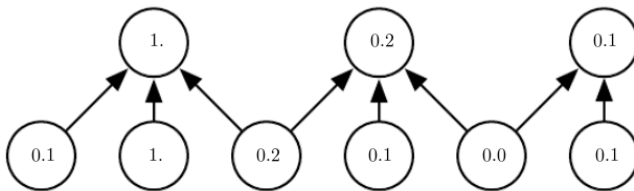
→ réponse identique aux deux entrées (rotation l'une de l'autre)

L'agrégation des sorties de plusieurs couches de convolution permet d'apprendre des invariants complexes



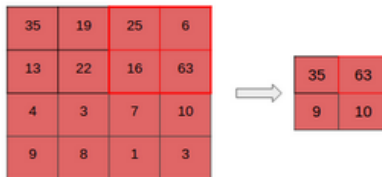
- Des alternatives à la convolution (« pure ») permettent justement de détecter des versions transformées d'un même feature
 - ▶ couche connectées localement
 - ▶ convolution tuilée

- L'agrégation spatiale « résumant » une carte d'activation, elle peut en diminuer la taille si on agrège avec un pas > 1 :
 - ▶ c'est un sous échantillonnage (sélectif) de la sortie convolutive



- Diminution du nombre de paramètres :
 - ▶ moins de calculs et de besoin mémoire
 - ▶ estimation statistique plus robuste (évite *over-fitting*)
- Peut permettre de ramener une donnée à une taille fixe avant MLP
- la fonction d'agrégation peut aussi être apprise [Jia et al, 2012]

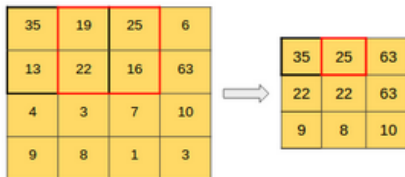
Agrégation (pooling) : exemple



Exemple de max pooling :

- avec *stride* [2 2]
- avec *stride* [1 1]

(iii)



Plan

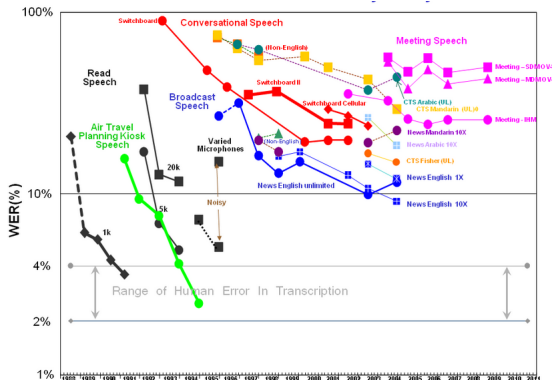
- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Campagnes d'évaluation

- Principe
 - ▶ Des organisateurs proposent un défi et des données d'entraînement
 - ▶ Chaque équipe développe (train+val) ses algorithmes
 - ▶ Les organisateurs publient des données de test non annotées
 - ▶ Chaque équipe calcule ses prédictions et les envoie aux organisateurs
 - ▶ Les organisateurs publient les résultats
- Avantages :
 - ▶ non intrusif : facilite la participation massive (→ stimulation mondiale)
 - ▶ évite sur-adaptation au corpus de test
- Inconvénients :
 - ▶ algos non transparents (sauf keynotes), risque de triche
 - ▶ biais vers la performance sur données/tâches particulières
- Différentes formes :
 - ▶ scientifiques : Trec(vid), TAC, CLEF, MediaEval...
 - ▶ autres : kaggle, privées...

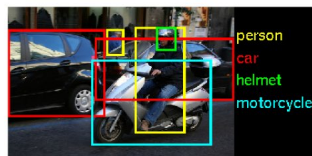
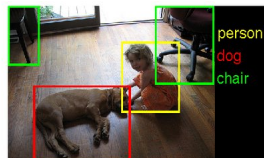
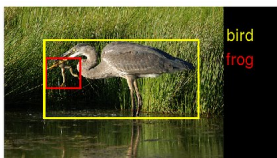
Campagnes sur la parole

- 1985 : NIST → 2002 : RT (*rich transcription*)
- En France, 1997 : AUPELF → 2003 ESTER (soutient DGA...)
- transcription (*speech2text*), journalisation (« qui parle quand ? »),
identification du genre/locuteur...

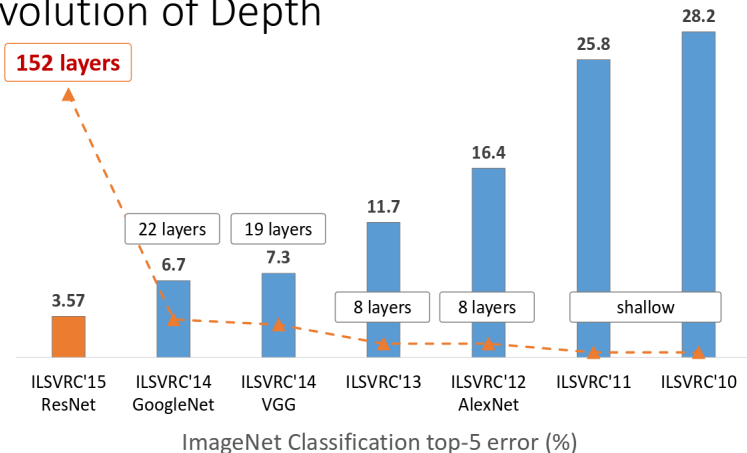


Challenge ILSVRC

- ImageNet Large Scale Visual Recognition Challenge (2010 – 2017)
- ImageNet :
 - ▶ 10M images classées dans +10k categories
 - ▶ classement hiérarchique (→ wordnet)
- *Localization & classification* : boîte autour des 5 principaux objets
- *Detection* : de 0 à bcp d'objets ; faux positifs pénalisés



Revolution of Depth



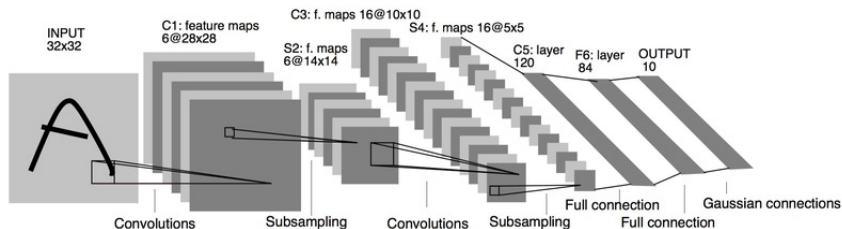
Kaiming He, Xiangyu Zhang, Shaoqing Ren, & Jian Sun. "Deep Residual Learning for Image Recognition". arXiv 2015.

Comment sommes-nous arrivés là ?

LeNet5

[LeCun et al., Proc. IEEE, 1998]

- 10 ans de travaux (1988 – 1998) appliqués à la reconnaissance de chèques de banque (écriture manuscrite)



Plusieurs bases du *deep learning* :

- Extraction feature par Conv et classification par MLP
- Couches conv+pooling+non-linéarité (sigmoïde ou tanh)
- Apprentissage par rétro-propagation

Implémentation GPU d'un MLP profond

- Des progrès en gestation (1998 – 2010)
 - ▶ Données disponibles en forte croissance (smartphones...)
 - ▶ Puissance de calcul (GPU → GPGPU en 2005)
 - ▶ Avancées théoriques [Hinton, 2007] [Hinton et al., 2006]
- Premier MLP profond implémenté sur GPU
 - ▶ MNIST : 60,000 images chiffre manuscrits
 - ▶ augmentation des données
 - ▶ Apprentissage accéléré $\times 40$ sur GTX280



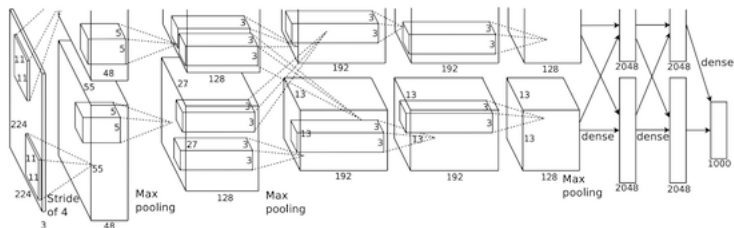
Approche	test error
pré-appr. non supervisé de RBM + KNN [Salakhutdinov & Hinton, 2007]	1.00
CNN + <i>elastic training image deformation</i> [Simard et al., 2003]	0.40
pré-appr. non supervisé de CNN + fine tuning [Ranzato et al., 2006, 2007]	0.39
MLP entraîné sur GPU [Ciresan et al., Arxiv 2010]	0.35

augmentation de données, RBM, pré-apprentissage : approfondi plus tard dans le cours

AlexNet

[Krizhevsky et al., NIPS 2012]

- LeNet-5, en plus grand, implémenté sur GPU
 - ▶ 60M paramètres dont $\sim 55M$ pour les *fully connected*
- Non linéarité ReLU
- Évite overfitting par dropout
- Grosse amélioration à ILSVRC 2012 → renouveau du deep learning!
- Implémentation CaffeNet disponible [Jia et al, Arxiv 2014]



dropout : approfondi dans cours 5

VGG

[Simonyan & Zisserman, Arxiv 2014, ICLR 2015]

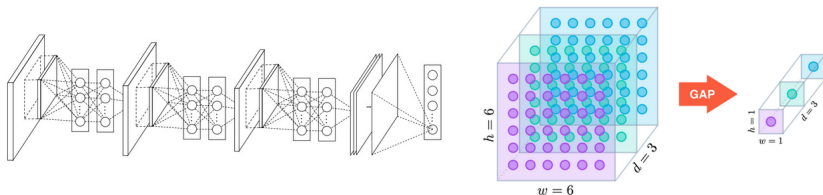
- *very deep* : passage à 16 ou 19 couches
- Multiples petits filtres au lieu de peu de grands
 - ▶ Plus de non linéarité → accroît la discrimination
 - ▶ décroît le nombre de paramètres à champ récepteur égal

	3×3	7×7
configuration	3×3	
	3×3	
champ récepteur	7×7	7×7
# param (mapping $C \rightarrow C$ canaux)	$3(3^2 C^2) = 27 C^2$	$7^2 = 49 C^2$

- Configurations de 11 à 19 couches
 - ▶ entraînement difficile : init avec 11 couches puis ajout successif
- Utilisation de conv $1 \times 1 \rightarrow$ NiN

Network In Network

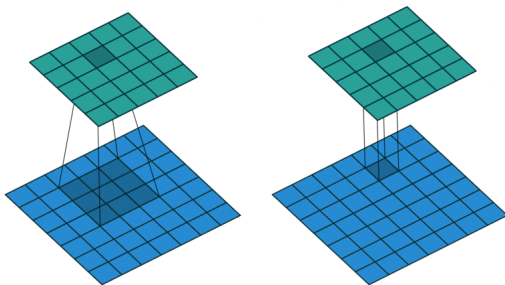
[Lin et al., Arxiv 2014]



- ajout d'un (petit) MLP *glissant* après chaque couche convolutive
- équivalent à convolution 1×1 (+ ReLU) voir slide suivante
- *global average pooling* sur les $N_{classes}$ derniers *feature maps*
 - ▶ remplace les fully connected (+aggrégation alternative au *max pooling*)
 - ▶ $N_{classes}$ *feature maps* en entrée du *softmax*
 - ▶ évite le risque d'overfitting du MLP (avg pool régularise « naturellement »)
 - ▶ meilleure correspondance entre catégories et features maps

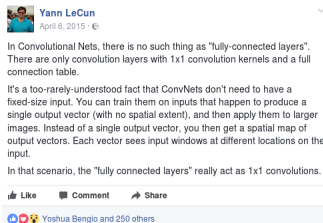
Convolution 1×1

Convolution 3×3 versus 1×1 :



- Conv. $1 \times 1 \times \text{C}_{input}$ $\rightarrow$ combinaison linéaire des feature maps d'entrée
- Une entrée de taille $N \times F \times H \times W$ convoluée avec filtre $1 \times 1 \times F_1$ donne une sortie de taille $N \times F_1 \times H \times W$.
 - ▶ $F_1 < F$: réduction de dimension
 - ▶ $F < F_1$: accroissement de dimension
- C'est un filtre $\rightarrow$ il est appris
 - ▶ réduction de dimension adaptée au réseau ($\neq$ PCA, sum pool...)

Convolution 1×1 : alternative (plus souple) aux FC

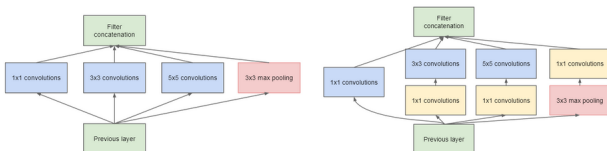


Full Connection Table : map chaque entrée à chaque sortie

- « FC sur entrée fixe » mathématiquement équivalent à convolution $1 \times 1 \times C_{input}$
- Implémentation souvent plus efficace
- en fin de réseau, peut être un classifieur « par pixel » $H \times W \times N_{classes}$
- indépendant de $H \times W$ donc $\neq$ FC qui nécessite un nb fixe d'entrées/sorties

Module inception (*bottleneck layer*)

[Szegedy *et al.*, Arxiv 2014]



- Problématique : vitesse en déploiement (reco visuelle chez Google)
- Solution : le « goulot » (*bottleneck*)
 - ▶ réduire le nombre de channels (avec conv 1×1)
 - ▶ convoluer
 - ▶ dilater le nombre de channels (avec conv 1×1)
- Ex : 256 entrées, conv 3×3 , 256 sorties (réduc. intermédiaire à 64)

calcul direct

$$256 * 3 * 3 * 256 \text{ MAC}$$

$$\rightarrow 590k \text{ MAC}$$

$$256 * 64 * 1 * 1 = 16k$$

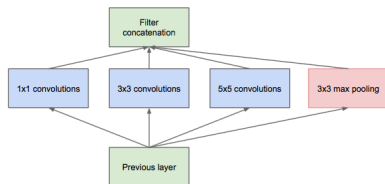
$$64 * 64 * 3 * 3 \sim 36k$$

$$64 * 256 * 1 * 1 = 16k$$

$$\rightarrow 70k \text{ MAC}$$

Module inception (*local sparsity*)

- un filtre convolutif permet de regrouper des informations
 - ▶ selon taille noyau on couvre des champs récepteurs de tailles variées
- Connexions parcimonieuses
désirables dans les ConvNets pour
raisons calculatoires (moins de paramètres...)
- en laissant le choix, on approxime la
structure parcimonieuse optimale
(qui existe en théorie [Arora *et al.*, 2013])
- garde des convolutions classiques pour invariance à la translation
- ajoute le pooling car « ça marche souvent bien »
- limité aux tailles 1×1 à 5×5 pour raisons pratiques
 - ▶ les 5×5 peuvent être remplacées par deux 3×3 (Inception v3 en 2016)
- « normalement » les modules larges doivent être plus choisis dans les
hautes couches (les plus abstraites)



GoogleNet

[Szegedy *et al.*, Arxiv 2014]

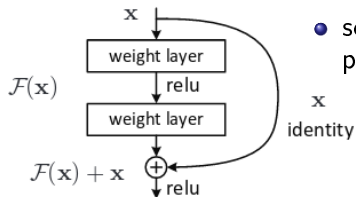


- Architecture basée sur des modules Inception
- Classifieurs intermédiaires pendant l'apprentissage
 - ▶ accroît le pouvoir discriminant pour les couches basses ?
 - ▶ accroît le gradient de retour (*BackProp*) cf. gradient évanescent ?
 - ▶ en fait [Szegedy *et al.*, 2016] :
 - ★ importance mineure, un seul suffit
 - ★ permet régularisation supplémentaire en fin de convergence

ResNet [He et al., 2015]

- Ajouter des couches à un NN (gain en profondeur) est :
 - ▶ bénéfique pour les perf. (mais convergence difficile)
 - ▶ erreurs (appr. et test) tendent à saturer... Puis se dégrader
- pb d'optimisation : difficulté à modéliser la partie identité avec de multiples non-linéarités

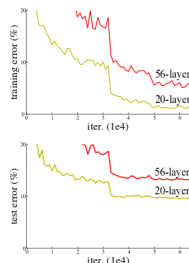
(non dû au gradient évanescent : v. cours 5)



- solution : **modélisation du résidu** (on ôte la partie linéaire en partie)

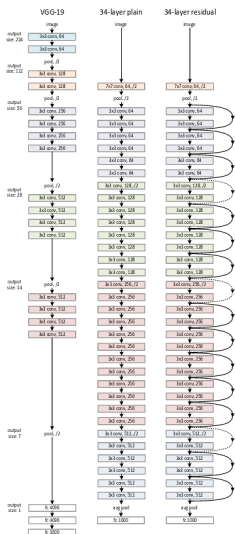
- ▶ on souhaite modéliser un mapping $\mathcal{H}(x)$ par quelques couches
- ▶ on modélise le résidu $\mathcal{F}(x) = \mathcal{H}(x) - x$
- ▶ puis ajouter x à $\mathcal{F}(x) = W_2 \sigma(W_1 x)$ ($\sigma = \text{ReLU}$)

exp. sur CIFAR-10

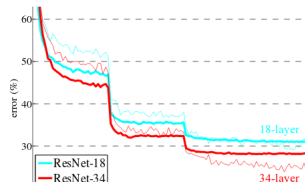
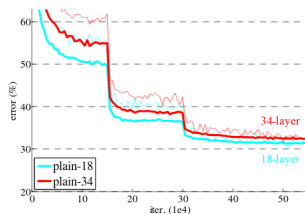


ResNet

[He et al., 2015]

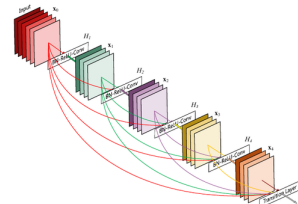
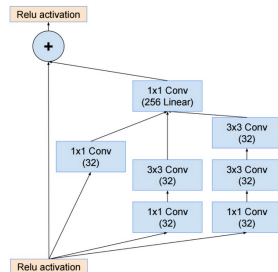
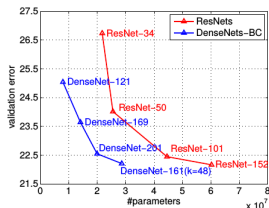


- ResNet a le même nombre de paramètres que sa version « pleine »
- Mais le ResNet profond a une erreur moindre

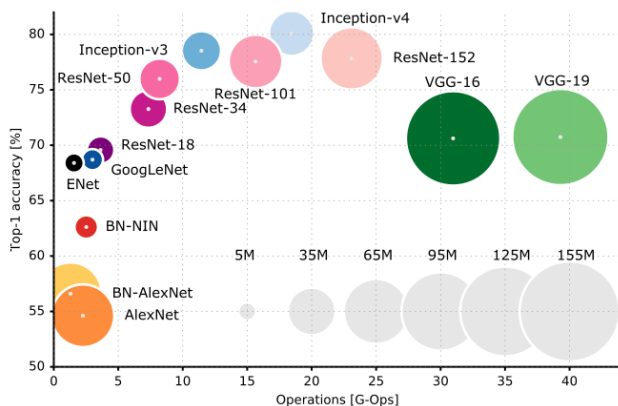
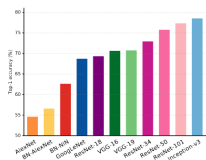


Extensions de ResNet

- Inception v4 [Szegedy *et al.*, 2016]
 - ▶ combine *Inception* et *ResNet*
 - ▶ marche bien... (mais difficile d'expliquer pourquoi)
- DenseNet [Huang *et al.*, 2016]
 - ▶ généralisation de ResNet avec des connexions à *toutes* les couches précédentes
 - ▶ adapte taille *feature map* par (average) *pooling*
 - ▶ moins de paramètres que ResNet mais perf. equiv.



Vue globale des architectures



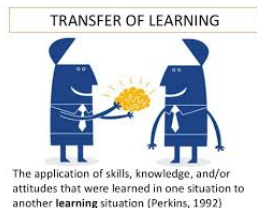
- abscisse : nombre d'opération en passe directe (*forward*)
- ordonnée : top-1 classif à ImageNet challenge
- taille blob $\sim$ nombre de paramètres

Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Gestion des problèmes d'intérêt réels

- Les CNN ayant des millions de paramètres, il faut beaucoup de données pour les apprendre
 - ▶ Typiquement : ImageNet = 1.2M images (1k classes)
- Problème : les problèmes cibles (ceux d'intérêt) ne comportent pas forcément beaucoup d'exemples (**annotation coûteuse**)
- Deux solutions principales :
 - ▶ Augmentation de données (*data augmentation*)
 - ▶ Transfert d'apprentissage (*transfer learning*)
- Une technique connexe
 - ▶ Réglage fin du réseau (*Fine tuning*)



Augmentation de données (*data augmentation*)

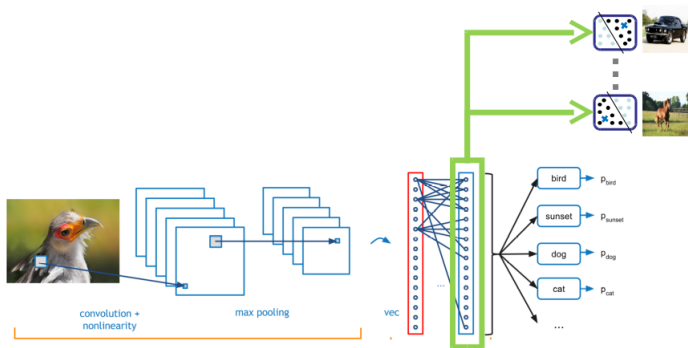
- Principe = accroître l'ensemble d'apprentissage en transformant les données (images), remplaçant par synonymes (texte), bruitant (son)...
- Ex : nombreuses transformations ne modifient pas la classe d'un objet
 - ▶ coût d'annotation nul (même label)
 - ▶ coût de collecte nul (img disponible + transfo. géométriques)



- Peut permettre une (relative) invariance à certaines transformations
- Certaines transformations ne sont pas toujours adaptées
- Plus de détails au **cours 5**

Transfert d'apprentissage (*transfer learning*)

- parfois impossible d'apprendre un CNN « from scratch » (i.e avec initialisation aléatoire) par manque de données (même avec data augmentation)
 - Ex : 10k images, transformées $\rightarrow$ 200k images $\ll$ 1.2M images
- On peut initialiser avec un réseau déjà appris (***pre-trained***)



Transfert d'apprentissage (*transfer learning*)

- Si le problème cible ressemble au dataset utilisé pour le pré-appris :
 - ▶ Si taille comparable : OK
 - ▶ Si taille très inférieure : risque *overfitting* après plusieurs passes du problème cible
- Pour petit problème → apprendre un classifieur linéaire (*one-vs-all*) avec une couche de haut niveau (e.g FC7) du réseau pré-appris
 - ▶ problème semblable → *feature* pertinent
- Si le problème cible est très différent du pré-appris
 - ▶ Si problème petit : classifieur linéaire avec couche de moins haut niveau
 - ▶ Si problème grand : *fine tuning*

Problèmes semblables ou différents ?

- expertise, connaissance métier... (pifomètre)
- un pré-apprentissage sur XXXXX est mieux que... Rien !

Réglage fin du réseau (*Fine tuning*)

- Si on a suffisamment de données on peut adapter les couches de haut niveau avec le problème cible
 - ▶ réinitialisation couche de sortie (e.g FC8) avec nouveau problème
 - ▶ mise à jour des poids ; pas forcément toutes les couches (e.g FC6 et FC7)
 - ▶ taux d'apprentissage plus faible qu'apprentissage problème source
- Le nombre de couches à adapter est fonction de :
 - ▶ la quantité de données
 - ▶ la ressemblance/dissembance des deux problèmes (cible et pré-appris)
- Peut être combiné à *data augmentation* et *transfer learning*
 - ▶ on peut réinitialiser plusieurs couches (e.g FC7 et FC8)

Exemple

On souhaite distinguer des modèles de voitures BMW (sans ex. d'appr.).

- on prend un réseau (VGG, GoogleNet, ResNet) appris sur ImageNet
- Transfert learning + Fine tuning avec une grande base de multiples marques de voitures
- Transfert learning (e.g SVM) sur les modèles BMW

Transfert de domaine

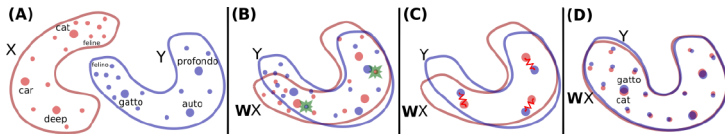
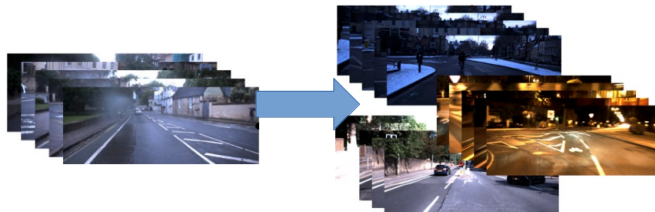
- Cas particulier de transfert d'apprentissage
 - ▶ avec les mêmes classes en source et en cible
 - ▶ mais données cibles ont une « apparence » différente de la source
 - ▶ *fine tuning* si suffisamment de données (+bcp d'autres méthodes)

En vision

[Wulfmeier *et al.*, 2017]

En NLP

[Conneau *et al.*, 2018]



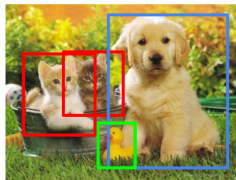
Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Détection

- But = trouver une boîte englobante autour des objets d'intérêt
- Comment poser le pb ?

credit : Li Fei-Fei



Régression : sortie taille variable



DOG, (x, y, w, h)
CAT, (x, y, w, h)
CAT, (x, y, w, h)
DUCK (x, y, w, h)

= 16 numbers



DOG, (x, y, w, h)
CAT, (x, y, w, h)

= 8 numbers



CAT, (x, y, w, h)
CAT, (x, y, w, h)
....
CAT (x, y, w, h)

= many numbers

Classification



CAT? NO

DOG? NO



CAT? YES!

DOG? NO

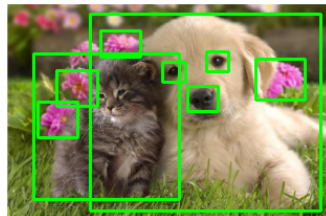


CAT? NO

DOG? NO

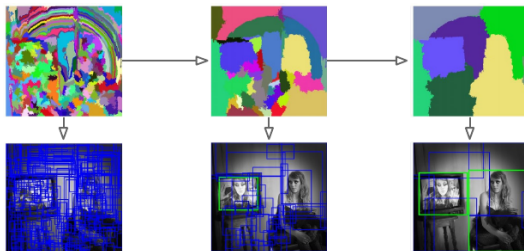
Détection

- Poser en tant que problème de classification ?
 - ▶ **Problème** : tester nombreuses positions et échelles (taille de boîte)
 - ▶ **Solution** : le faire si le classifieur est assez rapide
- Mais :
 - ▶ **Problème** : tester nombreuses positions et échelles avec CNN
 - ▶ **Solution** : sélectionner un petit ensemble des boîtes
- trouver des « blobs » susceptibles de contenir des objets
 - ▶ → détecteur d'objets agnostique à la classe (*objectness*)



Proposition de régions

- Recherche sélective (*selective search*)
 - ▶ segmentation ascendante (*bottom-up*) (Cf. cours vision)
 - ▶ fusionner les régions à différentes échelles
 - ▶ Convertir les régions en boîtes d'intérêt potentiel

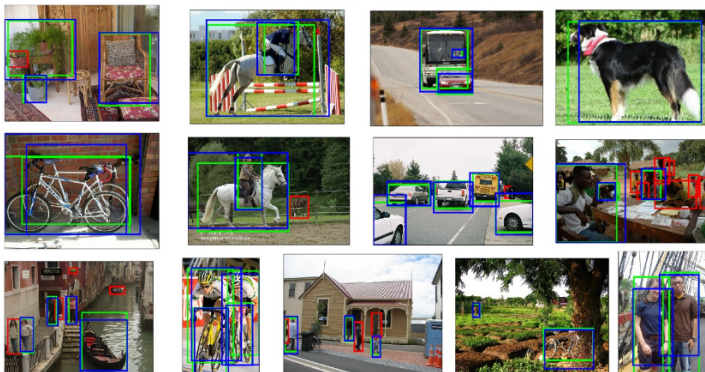


[Uijlings et al, « Selective Search for Object Recognition », IJCV 2013]

Proposition de régions

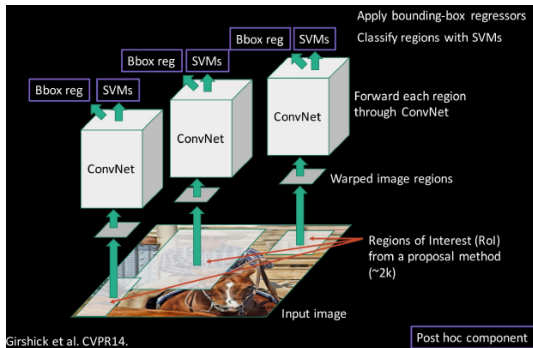
• EdgeBoxes

- ▶ le nombre de contours contenus dans une boîte est un bon indicateur de la vraisemblance qu'il contienne un objet
- ▶ score *objectness* $\propto$ nb bords centraux - nb bords latéraux
- ▶ densité de proposition fonction de la qualité ($\frac{\text{intersection}}{\text{union}}$) désirée
- ▶ résultats (GT = vert ou rouge ; proposition EdgeBoxes en bleu)



R-CNN

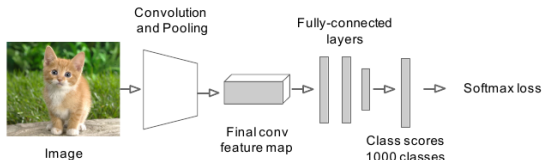
R-CNN = proposition de région + CNN + SVM linéaire



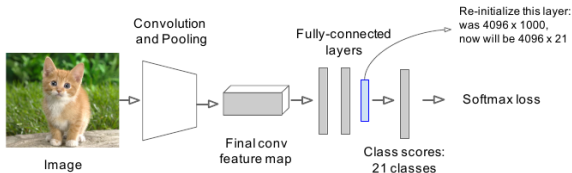
[Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014]

Apprentissage R-CNN (1/3)

- (pré)apprentissage CNN (AlexNet, VGG, GoogleNet...) sur ImageNet (1.2M images, 1k classes)

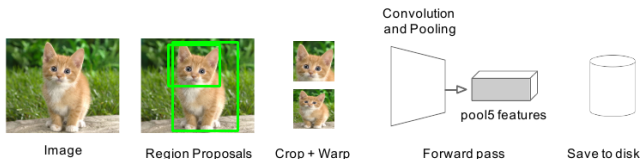


- transfer learning + fine tuning pour les classes d'intérêt (+fond)
 - ▶ on ôte la dernière couche et on réapprend sur problème d'intérêt

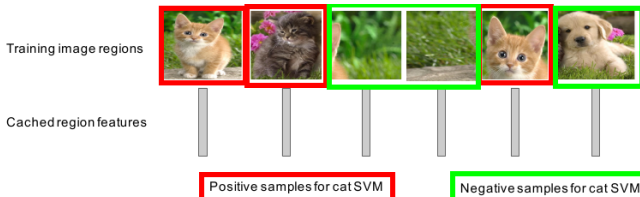


Apprentissage R-CNN (2/3)

- propositions d'images e.g avec *selective search* pour chaque région puis :
 - ▶ retailler chaque région au format d'entrée du CNN
 - ▶ sauvegarder la dernière sortie convolutive



- Apprendre un SVM binaire (2 classes) pour chaque classe d'intérêt (feature = dernière sortie convolutive)



Apprentissage R-CNN (3/3)

- Pour chaque classe, apprendre une régression linéaire permettant de corriger des petits « décalages »

Training image regions



Cached region features



Regression targets
(dx, dy, dw, dh)
Normalized coordinates

(0, 0, 0, 0)
Proposal is good



(.25, 0, 0, 0)
Proposal too
far to left



(0, 0, -0.125, 0)
Proposal too
wide

Limites de R-CNN

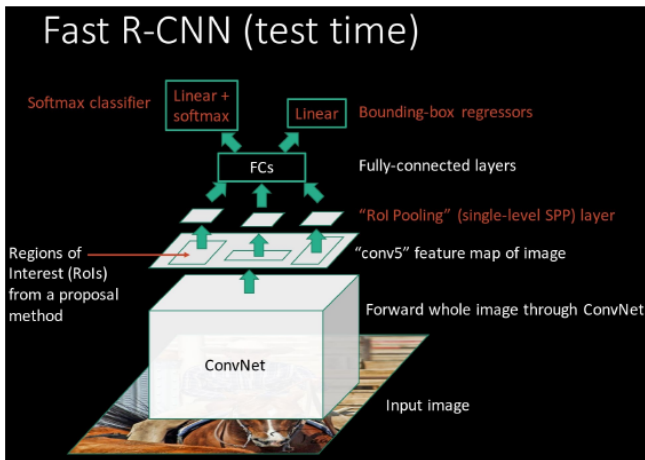
- Assez lent en phase de test : passe directe dans le CNN pour *chaque* proposition de région (≈ 2000)
- Les SVMs et la régression sont un peu « ad hoc »
 - ▶ on pourrait d'ailleurs se passer des SVMs en utilisant directement la sortie du CNN adapté aux classes. Du fait de la définition prise pour les exemples pos/neg, les SVMs fonctionnent mieux...
- L'apprentissage est assez complexe (plusieurs étapes...)

Deux autres méthodes :

- Fast R-CNN [Girschick, ICCV 2015]
- Faster R-CNN [Ren *et al.*, NIPS 2015]

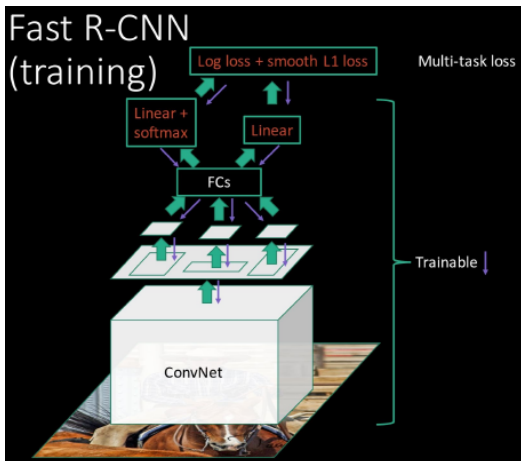
Fast R-CNN

- **Problème** : lent en phase de test car passes directes de chaque région
- **Solution** : un seule passe de l'image en entier (*ROI pooling* à « conv5 » (détaillé plus loin))



Fast R-CNN

- **Problème** : apprentissage ad-hod + complexe
- **Solution** : un seul système CNN intégrant le tout !

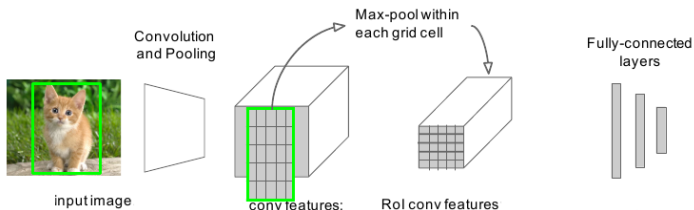


- coût multitâche
 - ▶ classif : coût par M.V + sortie sigmoïdale
 - ▶ régression par $smooth_{L_1}(x)$:

$$S_{L_1} = \begin{cases} 0.5x^2 & \text{si } |x| < 1 \\ |x| - 0.5 & \text{sinon} \end{cases}$$

Fast R-CNN : *Region of Interest Pooling*

- **Problème** : les couches FC ont besoin d'une taille fixe $h \times w \times C$ en entrée alors que les features des régions ont des tailles variées $H \times W \times C$
- **Solution** :
 - ▶ projeter la proposition de région dans la dernière feature map convolutive
 - ▶ diviser la région en une grille $h \times w$
 - ▶ max-pool dans chaque cellule de la grille



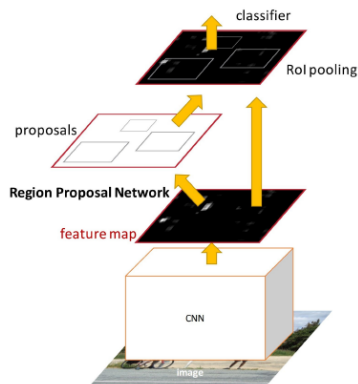
Fast R-CNN : résultats et limites

	R-CNN	Fast R-CNN
Test time per image	47 seconds	0.32 seconds
(Speedup)	1x	146x
Test time per image with Selective Search	50 seconds	2 seconds
(Speedup)	1x	25x

- **Problème** : les résultats ci-dessus (en test) n'incluent pas la phase de proposition de région
- **Solution** : inclure la proposition de région dans le CNN !

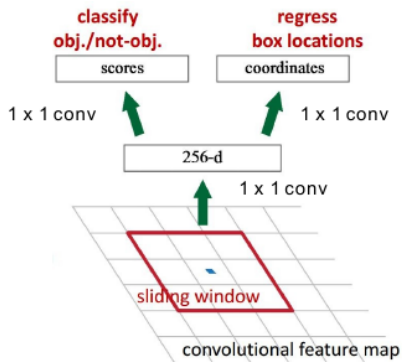
Faster R-CNN

- Élément clé : (RPN *region proposal network*)
- Un réseau appris pour faire des propositions de région
- Inséré après la dernière couche convolutive
- Le reste est semblable à Fast-RCNN



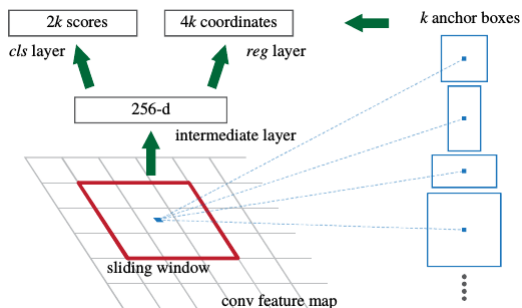
Faster R-CNN : region proposal network

- fenêtre glissante sur la *feature map*
- petit réseau pour
 - ▶ classer en objet/non-objet
(softmax 2 classes)
 - ▶ régression sur la localisation
- la position de la fenêtre glissante = information grossière par rapport à l'image originale (sous échantillonnage)
- la régression donne une localisation plus fine par rapport à la fenêtre glissante



Faster R-CNN : region proposal network

- définition de $k = 9$ ancres (*anchors*)
 - ▶ 3 échelles (*scale*)
 - ▶ 3 formes de régions (*aspect ratio*)
- à chaque localisation de la fenêtre, on prédit k propositions
- la régression donne l'offset par rapport à chaque « ancre »
- la classification donne la probabilité que l'« ancre régressée » soit un objet



Faster R-CNN : résultats

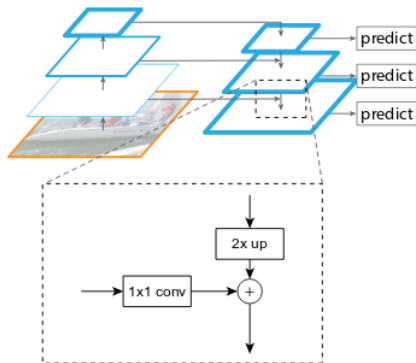
	R-CNN	Fast R-CNN	Faster R-CNN
Test time per image (with proposals)	50 seconds	2 seconds	0.2 seconds
(Speedup)	1x	25x	250x
mAP (VOC 2007)	66.0	66.9	66.9

- État de l'art début 2016 (combiné à ResNet)
 - ▶ toujours proche état de l'art en 2019, mais lent (cf. suite)
- Code disponible : (matlab), (py)Caffe, tensorflow, pyTorch...
- Mask R-CNN (He et al., ICCV 2017) : extension de Faster R-CNN à la segmentation sémantique

Feature Pyramid Networks (Lin et al., CVPR 2017)

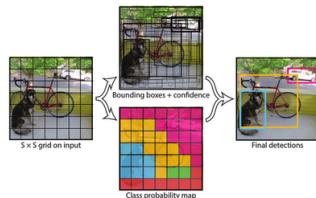
- Double pyramide montante (BU) et descendante (TD)
 - ▶ bottom-up (BU) = CNN classique : conv de résolution décroissante
 - ▶ top-down : résolution croissante enrichie par couche BU correspondante

- basses résolution moins précises mais plus « fortes sémantiquement »
- architecture générique pouvant être intégré à e.g Fast R-CNN ou RPN



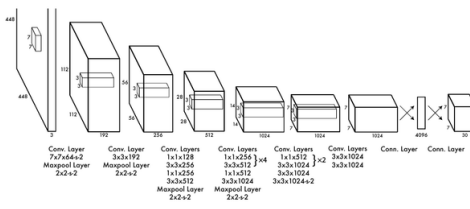
YOLO : *you only look once* (Redmond et al., 2016)

- Problème de détection reformulé en une unique régression pour la reconnaissance ET la localisation
- Premier système temps réel avec performances en reconnaissance proches de Faster R-CNN
- Découpage img en e.g 7×7 cellules
- Chacune détermine conjointement *objectness* et *box*
 - ▶ objet dont centre est dans la cellule
 - ▶ box peut être au delà de la cellule
 - ▶ confiance = $Pr(Object) * IoU_{pred}^{truth}$
- Chacune détermine $Pr(class_i | Object)$ pour classes d'intérêt
- En test, le produit des deux donne un score pour chaque classe dans chaque *box*



YOLO : *you only look once* (Redmond et al., 2016)

- Architecture à 24 couches convolutives (9 pour *fast YOLO*)
 - Pré-entraîné en 224 sur la classification ImageNet (une semaine...)
 - Détection utilise résolution 448×448

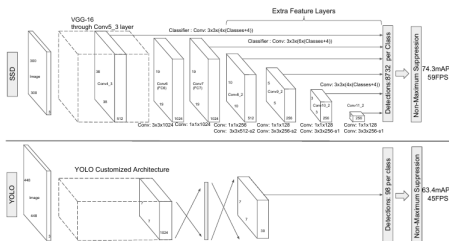


- minimisation aux moindres carrés
- les *boxes* sans objets biaisent le loss
 - $\rightarrow \lambda_{coord} = 10 \lambda_{noobj}$
- une seule box par cellule responsable de prédire un objet donné ($\max_{loU}$) : $\mathbb{1}_{ij}$ (cell i ; box j)

$$\begin{aligned}
 & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 \\
 & + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
 \end{aligned}$$

SSD : *single shot multibox detector* (Liu et al., 2016)

- Architecture purement convolutive
 - ▶ Réseau « de base » (VGG-16) et *conv* supplémentaire
 - ▶ Chaque *conv* est analysée par un réseau conv 3×3 prédisant en chaque point :
 - ★ quatre coordonnées de localisation
 - ★ la probabilité de présence de chaque classe
 - ▶ *non max suppression* pour garder les boîtes significatives
- coût = softmax + $smooth_{L_1}$ (cf. Fast R-CNN)



- Boîtes à plusieurs ratio (1, 2, 3, 1/2, 1/3) ajustées
- Augmentation de données
- → performances de Faster R-CNN et rapidité de YOLO

YOLOv2 (Redmond et Farhadi, 2017)

- Analyse des erreurs de YOLO → multiples améliorations :
 - normalisation des batches (Cf. cours 6)
 - pré-entraînement de la classif. à plus grande résolution ($224 \times 224 \rightarrow 448 \times 448$)
 - intégraton de la RPN de FasterRCNN
 - dim. région choisie par clustering
 - prédiction directe de la localisation → système plus stable
 - descripteurs de région plus fins (concat. $13 \times 13 \rightarrow 26 \times 26$)
 - entraînement avec des images multi-échelles (320 à 608 pix)
- amélioration de 15pt mAP sur PascalVOC07 !
- Compromis FPS/mAP
 - 40/78.6 sur 544×544
 - 40/63.4 pour YOLO
 - 67/76.8 sur 416×416

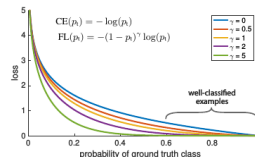
	YOLO									YOLOv2
batch norm?		✓	✓	✓	✓	✓	✓	✓	✓	
hi-res classifier?	●		✓	✓	✓	✓	✓	✓	✓	
convolutional?				✓	✓	✓	✓	✓	✓	
anchor boxes?				✓						
new network?					✓				✓	
dimension priors?						✓	✓	✓	✓	
location prediction?	●					✓	✓	✓	✓	
passthrough?							✓		✓	
multi-scale?								✓	✓	
hi-res detector?		+3.7			+4.8				✓	
VOC2007 mAP	63.4	65.8	69.5	69.2	69.6	74.4	75.4	76.8	78.6	

RetinaNet (Lin et al., 2017)

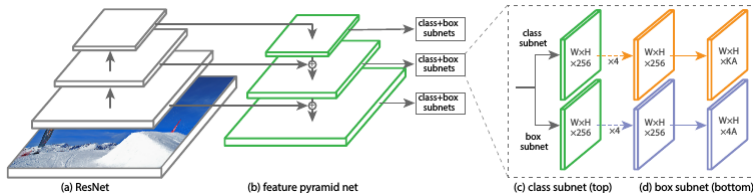
- Les archis à 1 étape génèrent de nombreux échantillons négatifs (de fond) (trop) faciles à classer → biaise le modèle

Le *Focal loss* décroît l'importance des exemples faciles à classer ($p_t = p$ si $y = 1$ et $1 - p$ sinon) :

$$FL(p_t) = -(1 - p_t)^\gamma \log(p_t)$$



- Archi ResNet/FPN et deux sous réseaux indépendants ($\neq$ RPN) :
 - classification (out = sigmoïde) : réseau convolutif (multiples conv 3×3)
 - localisation (out = linéaire) : idem



Bilan des systèmes de détection d'objets

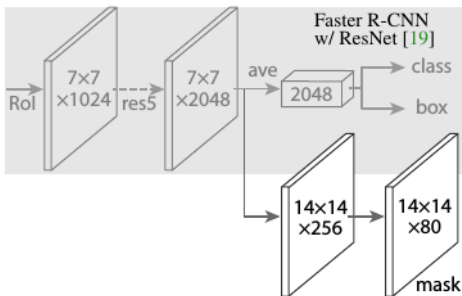
Deux familles d'approche :

- méthodes « à deux étages » (*two stages*), *i.e* avec proposition de région
 - ▶ R-CNN : système « de base »
 - ▶ Fast R-CNN : intègre la régression et la classification
 - ▶ Faster R-CNN : proposition de région intégrée
 - ▶ Auto-ML (+ Faster R-CNN) : optimisation (?) des *features*
- méthodes « à un étage » (*one stage*) ou « à échantillonnage dense »
 - ▶ YOLO : régression unique pour localisation et reconnaissance
 - ▶ SSD : que des convolutions
 - ▶ YOLOv2 : multiples améliorations
 - ▶ RetinaNet : corrige le biais dû aux nombreux exemples négatifs

Segmentation sémantique

[He et al., Mask R-CNN, ICCV 2017]

- problème de classification en chaque pixel d'une image
- Ajout d'une 3^{ème} branche à Faster-RCNN pour prédire masque
 - ▶ Effectue aussi détection (box) et classification
 - ▶ Multi-task loss (somme des trois)



- Peut utiliser différent *backbones* ; ResNet, FPN...

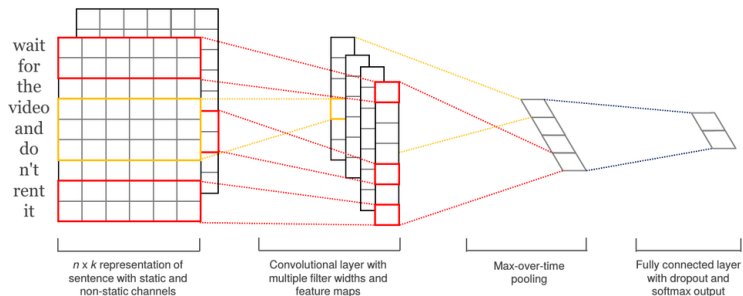
Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 **Autres architectures convolutives**
 - Architectures pour la détections d'objets
 - **Représentation textuelle par CNN**
- 7 Conclusion

CNN pour la représentation textuelle

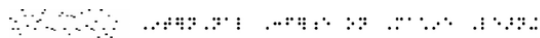
[Kim, EMNLP 2014] [Zeng *et al*, COLING 2014]

- Un N-gram (cf. cours 2) est un ensemble ordonné de N mots
- C'est un *pattern* détectable par un noyau convolutif
 - ▶ taille filtre (1D) = nombre de mots pris en compte
- Représentation de chaque mot par vecteur dense (*embedding*)



CNN pour la représentation textuelle

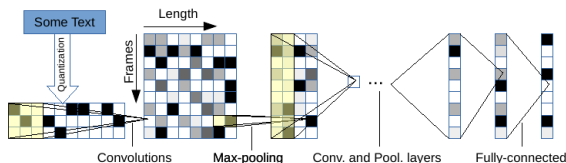
- Apprendre à partir des caractères [Zhang & LeCun, 2016]
 - ▶ alphabet taille $m(=70)$, séquences de caractères de longueur l
 - ▶ codage binaire de chaque caractère (1 parmi m) : similaire Braille



(a) Binary

(b) Braille

- (*data augmentation* avec des synonymes)
- Modèle = 6 conv + 3 FC ; dernière couche dépend du problème
 - ▶ *text classif* : nombre de classes (analyse de sentiments, news categ....)



Plan

- 1 Introduction
- 2 Convolution
 - Définition
 - Propriétés remarquables
 - Couches convolutives
- 3 Agrégation (pooling)
- 4 Architectures CNN classiques pour la classification
- 5 Transfert d'apprentissage et augmentation de données
- 6 Autres architectures convolutives
 - Architectures pour la détections d'objets
 - Représentation textuelle par CNN
- 7 Conclusion

Conclusion

- Opération de convolution : définition, calcul pratique...
- Couches de convolution et leurs propriétés
 - ▶ interactions parcimonieuses → calcul efficace
 - ▶ partage de paramètres → robustesse statistique (+gain mémoire)
 - ▶ équivariance de représentation → invariance en translation
- propriétés du pooling
- Architectures pour la classification : AlexNet, VGG, GoogleNet, ResNet
- Architectures pour la détection : R-CNN, Fast-R-CNN, Faster-R-CNN (+RPN!)
- Gestion des problèmes réels :
 - ▶ augmentation de données
 - ▶ Transfert d'apprentissage (*transfer learning* !)
 - ▶ Réglage fin (*fine tuning*)

Références

- Canziani A., Paszke A., Culurciello A. (2016) An Analysis of Deep Neural Network Models for Practical Applications, ArXiv 1605.07678
- Ciresan D.C., Ueli Meier, Luca Maria Gambardella, Juergen Schmidhuber (2010) « Deep Big Simple Neural Nets Excel on Handwritten Digit Recognition », arXiv :1003.0358
- He K., Zhang X., Ren S. and Sun J. (2015) Deep Residual Learning for Image Recognition, arXiv :1512.03385
- Hinton, G. E., Osindero, S., and Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. Neural Computation, 18.
- Hinton, G. (2007). To recognize shapes, first learn to generate images. Computational Neuroscience : Theoretical Insights into Brain Function., Elsevier.
- Krizhevsky A., Ilya Sutskever, Geoffrey E. Hinton (2012) ImageNet Classification with Deep Convolutional Neural Networks, Advances in Neural Information Processing Systems 25 (NIPS 2012)
- Jia Y., E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, T. Darrell, (2014) Caffe : Convolutional architecture for fast feature embedding, arXiv :1408.5093.
- Le Cun Y., L. Bottou, Y. Bengio, and P. Haffner (1998) Gradient-based learning applied to document recognition. Proceedings of the IEEE, november 1998.
- Liu W., Anguelov D., Erhan D., Szegedy C., Scott Reed, Cheng-Yang Fu, and Alexander C. Berg (2016) SSD : Single Shot MultiBox Detector, ECCV

Références

- Conneau, A., Lample, G., Ranzato, M., Denoyer, L., & Jégou, H. (2018). Word Translation Without Parallel Data. ICLR
- Kim Y.(2014) Convolutional Neural Networks for Sentence Classification. EMNLP. 2014.
- Lin M., Qiang Chen, Shuicheng Yan (2014) Network In Network, ICLR 2014
- Ranzato, M., Huang, F., Boureau, Y. & LeCun, Y.(2007). Unsupervised Learning of Invariant Feature Hierarchies with Applications to Object Recognition. CVPR'07
- Ranzato, M., Poultney, C., Chopra, S. & LeCun, Y. (2006). Efficient Learning of Sparse Representations with an Energy-Based Model. NIPS 2006.
- Redmon J., Divvala S., Girshick R., Farhadi A. (2016) You Only Look Once : Unified, Real-Time Object Detection, CVPR, Dallas, USA
- Salakhutdinov, R. & Hinton, G. (2007). Learning a Nonlinear Embedding by Preserving Class Neighborhood Structure. AISTATS, 11
- Sanjeev Arora, Aditya Bhaskara, Rong Ge, and Tengyu Ma. Provable bounds for learning some deep representations. CoRR, abs/1310.6343, 2013

Références

- Simard, P.Y., Steinkraus, D. & Platt, J.C. (2003). Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis. ICDAR 2003, pp 958 – 962.
- Simonyan K. & Zisserman A. (2014), Very Deep Convolutional Networks for Large-Scale Image Recognition arXiv technical report, 2014
- Sermanet P., Eigen D., Zhang X., Mathieu M., Fergus R., Le Cun Y. (2013) OverFeat : Integrated Recognition, Localization and Detection using Convolutional Networks. arXiv :1312.6229
- Steinkraus D., Ian Buck et Patrice Simard, (2005) « Using GPUs for Machine Learning Algorithms » ICDAR 2005, pp. 1115–1119
- Szegedy C., W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich (2014) Going Deeper with Convolutions, arXiv :1409.4842v1
- Szegedy C., Ioffe S., Vanhoucke V., Alemi A.(2016) Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning, arXiv :1602.07261
- M. Wulfmeier, A. Bewley, and I. Posner (2017) Addressing Appearance Change in Outdoor Robotics with Adversarial Domain Adaptation, in IEEE/RSJ International Conference on Intelligent Robots and Systems
- Zeng D. et al (2014) Relation Classification via Convolutional Deep Neural Network. COLING. 2014.